RAG Assignment: Baseline vs. Re-Ranking

Sohail Memon
Roll No.: 2025201075

Document: RFC 6749, *The OAuth 2.0 Authorization Framework* (76 pages).
Code: https://github.com/SohailTSM/Rag-Bi-Encoder-vs-Cross-Encoder-Reranking

1 Architecture comparison

The main difference is *when the question and the chunk meet.* In a bi-encoder they never meet inside the model. In a cross-encoder they are read together (Figure 1).

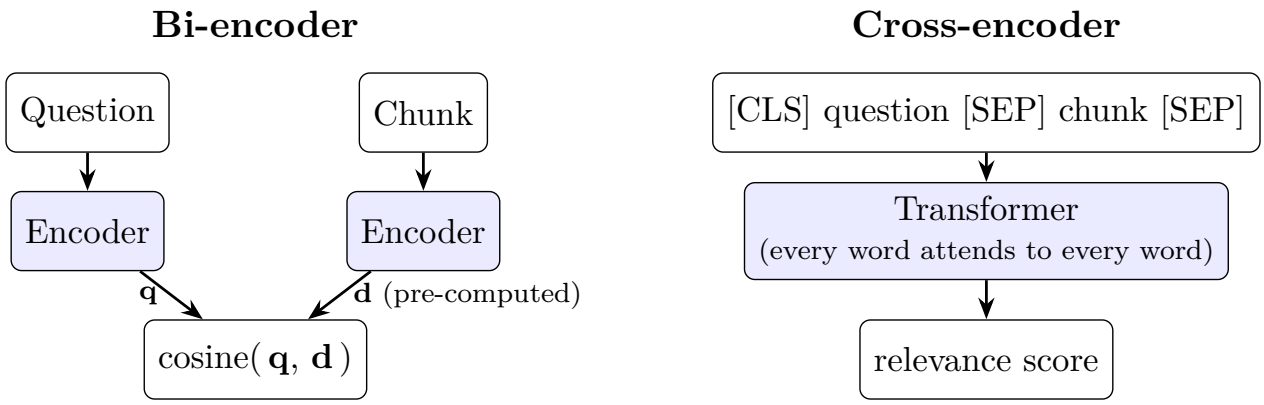


Figure 1: A bi-encoder encodes the question and the chunk separately and compares two vectors. A cross-encoder reads both together and outputs one score.

Bi-encoder (`all-MiniLM-L6-v2`). The question and each chunk go through the encoder separately. Each one becomes a single 384-number vector, and the score is the cosine between the two vectors. Because the chunks don't depend on the question, their vectors can be computed once in advance and stored in a vector index (I used FAISS). At question time only the question is encoded. The weak point is that the whole chunk is squeezed into one vector before the model has seen the question. Small but important things, like a "not" or which grant type a rule is about, get averaged away. In my runs the scores of the 10 candidates were very close to each other (for question N1 all 10 were between 0.65 and 0.70), so small word overlaps decided the order.

Cross-encoder (`ms-marco-MiniLM-L-6-v2`). The cross-encoder takes the question and the chunk as one input, `[CLS] question [SEP] chunk [SEP]`. Inside the transformer, self-attention lets every word of the question look at every word of the chunk, in every layer, and at the end it gives one relevance score. Because of this it can notice that "implicit grant" + "refresh token" + "does not support" in one chunk answers the question. Its scores are also much more spread out (for N1 the right chunk got 7.04 and the next one only 3.43). The weak point is that nothing can be computed in advance: every (question, chunk) pair needs one full pass through the model.

2 Qualitative and rank shift analysis

For each test question I marked by hand the chunk that holds the answer (the "right chunk"). Table 1 shows where that chunk was ranked by the bi-encoder (Pipeline 1) and after cross-encoder re-ranking (Pipeline 2). Only the top 3 chunks are given to the LLM.

ID	Failure type	Question	Right chunk	Rank change
K7	keyword trap	Should a browser or proxy be allowed to store the token response?	§5.1	#6 → #1
E5	exception	Can the server hand out a new refresh token when the client refreshes, and what happens to the old one?	§6	#4 → #1
N1	negation	Is the implicit grant allowed to issue a refresh token?	§4.2	#5 → #1
N7	negation	Which clients are not allowed to use the password grant?	§1.3.3, §4.3	#6 → #1
B7	buried (rank 4-10)	What should the server do if the redirect URI doesn't match the registered one?	§4.1.2.1, §3.1.2.4	#7 → #3
K3	keyword trap	How can a website stop another site from forging the callback to its redirect page?	§10.12	#3 → #4
N9	negation	Are public clients required to authenticate at the token endpoint?	§2.3	#1 → #7

Table 1: Rank of the right chunk before and after re-ranking. The first five were fixed by re-ranking; the last two (K3, N9) were made worse.

Answer 1 (baseline) vs. Answer 2 (re-ranked)

Both answers were generated locally with `qwen2.5:7b-instruct` using the same prompt; only the three context chunks are different.

K7 - baseline could not answer.

> **Answer 1:** *"The provided context does not answer this question."*
>
> **Answer 2:** *"MUST NOT, as the authorization server MUST include the HTTP 'Cache-Control' response header field with a value of 'no-store' in any response containing tokens. (Section 5.1)"*

The rule was at rank 6, so the baseline LLM never saw it. The baseline chunks (§4.2.2 and a part of §5.1) share the words "token response" with the question but say nothing about caching.

E5 - baseline misstated the rule and left out a clause.

> **Answer 1:** *"The server can hand out a new refresh token when the client refreshes, and the old refresh token is typically invalidated but retained by the authorization server."*
>
> **Answer 2:** *"Yes, the server can hand out a new refresh token when the client refreshes. In this case, the client must discard the old refresh token and replace it with the new one. The server MAY revoke the old refresh token after issuing a new one. (Section 6)"*

Answer 1 took an optional example from the security section ("could employ refresh token rotation") and presented it as what "typically" happens. It also missed the actual rule that the client must discard the old token.

N1 - negation fixed.

> **Answer 1:** *"The provided context does not answer this question."*
>
> **Answer 2:** *"The implicit grant type is not allowed to issue a refresh token. (Section 4.2)"*

The bi-encoder ranked four general refresh-token chunks above the one sentence that carries the "not".

B7 - partial answer became a full answer.

> **Answer 1:** *"The server should not proceed with the authorization request ... The authorization server MUST validate the provided redirect URI against the registered value (Section 10.6)."*
>
> **Answer 2:** *"... If there is a mismatch, the server MUST NOT automatically redirect to the invalid URI and SHOULD inform the resource owner of the error instead. (Section 3.1.2.3)"*

Answer 1 knows the URI must be checked, but not what to do when it fails ("should not proceed" is its own guess). Answer 2 has the real rule, though it cites the wrong section (the sentence is from §4.1.2.1).

N7 - retrieval fixed, but the LLM still refused. Both answers were "The provided context does not answer this question." After re-ranking the right chunks were at ranks 1 and 2, but they only say the password grant should be used "when there is a high degree of trust". The RFC never lists clients that are "not allowed", so the model refused.

K3 and N9 - re-ranking made the answer worse. For N9 the baseline answer was correct: *"Public clients are not required to authenticate at the token endpoint. The authorization server MAY establish a client authentication method with public clients but MUST NOT rely on public client authentication ... (Section 2.3)"*. After re-ranking, §2.3 dropped to rank 7 and Answer 2 became *"The provided context does not answer this question."* For K3, Answer 1 had the real requirement (*"The client MUST implement CSRF protection for its redirection URI ... "*, §10.12), while Answer 2 only said to use the `state` parameter. In both cases the cross-encoder preferred chunks that repeat the question's words ("client authentication", "cross-site request forgery") over the chunk with the actual rule.

Did the baseline hallucinate? No. In my runs the baseline never made up a fact. When the right chunk was outside the top 3, it either said it could not answer (K7, N1) or omitted and misstated a rule (E5, B7). I think this is because my prompt tells the model to answer only from the context.

3 Latency profiling

I timed each step with `time.perf_counter()` for 34 test questions, 5 times each, after a warm-up run, on the GPU (RTX 5050 laptop) and on the CPU.

Step	GPU: median (p95), ms	CPU: median (p95), ms
Encode the question (bi-encoder)	2.84 (4.04)	7.87 (8.36)
FAISS top-10 search	0.03 (0.05)	0.14 (0.15)
Cross-encoder on 10 pairs	14.06 (17.39)	131.87 (170.05)
Baseline retrieval (Pipeline 1)	**2.88 (4.09)**	**8.01 (8.50)**
Re-ranking pipeline (Pipeline 2)	**17.06 (20.44)**	**139.92 (177.63)**
Extra time for re-ranking	+14.2 (5.9×)	+131.9 (17.5×)

Table 2: Retrieval runtime per question.

Re-ranking makes retrieval about 6 times slower on the GPU and about 17 times slower on the CPU. Almost all of the extra time is the cross-encoder.

4 Scaling question

Why don't we run the cross-encoder over the whole collection from the start? Because its cost grows with the number of documents, and nothing can be computed in advance. I measured this by running the cross-encoder on all 228 chunks for each question: it took 247 ms on the GPU (about 1.08 ms per pair) and 3.3 s on the CPU. The bi-encoder with FAISS searched the same 228 chunks in 3.3 ms. Using the cost per pair, I estimated bigger collections:

Chunks	CE on all (GPU)	CE on all (CPU)	CE on top 10 only (GPU / CPU)
228 (this RFC)	0.25 s	3.3 s	14 ms / 132 ms
10,000	10.8 s	2.4 min	same
1,000,000	18.1 min	4.0 h	same
100,000,000	30.1 h	404 h	same

Table 3: Estimated time per question (linear extrapolation; CE = cross-encoder).

- The cross-encoder needs one full transformer pass for every (question, chunk) pair. Every score depends on the question, so nothing can be saved or cached, and the cost grows linearly with the number of documents. At one million chunks, one question would take about 18 minutes even on my GPU.
- The bi-encoder moves the expensive part offline. All chunks are encoded once, and at question time only the question is encoded, followed by a fast vector search (with an approximate index like HNSW this stays in milliseconds even for millions of documents).
- So the two-stage design uses the cheap model to cut the collection down to 10 candidates, and the expensive model only re-orders those 10. Its cost stays about 14 ms no matter how big the collection is.
- The price is recall: the re-ranker can't fix what the first stage misses. In 4 of my 34 questions the right chunk was not in the top 10, so re-ranking could not help.